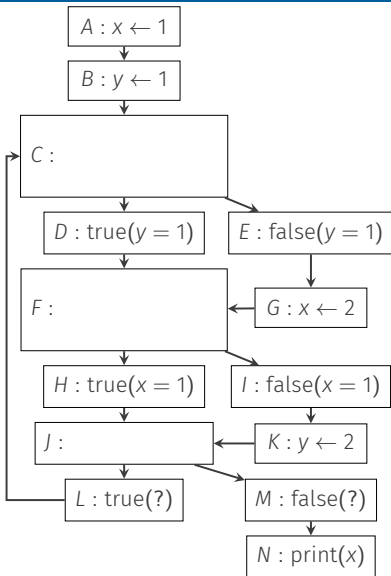


Static Single Assignment Form

Roland Leißa (based on slides by S. Hack)

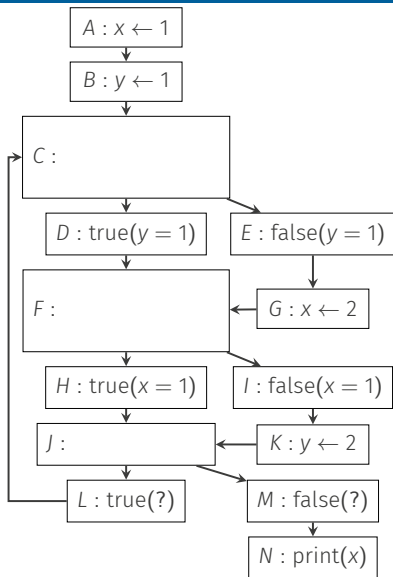
University of Göttingen

Problem and Motivation



- Consider Constant Propagation
- Lattice:
$$V \rightarrow (Vars \rightarrow \mathbb{Z}_{\perp}^T) \times \{\perp_{UR}, \top_{UR}\}$$
- Per CFG node we have to keep a mapping from $|Vars|$ variables to abstract values
- Space requirement $V \times |Vars|$

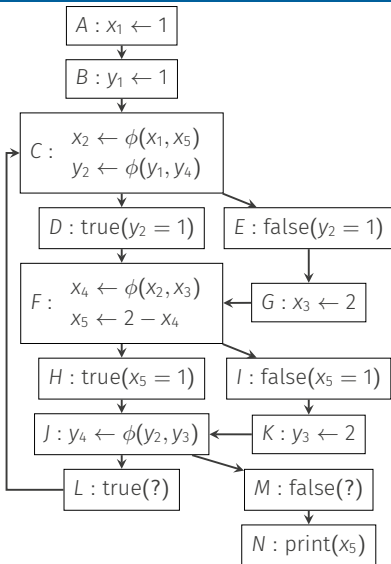
Flow-Insensitive Constant Propagation



- Get around storing a map from vars to $\mathbb{Z}_{\perp}^T$ at every program point
- Keep one element $x \in \mathbb{C}$ per CFG not per program point
- Solve the single equation

$$d \sqsupseteq \bigsqcup_i f_i^{\#}(d)$$

- Loss of precision because abstract values of all definitions of a variable are joined



- Flow-Insensitive Analyses
- Each Variable has a **static single assignment**, i.e. one program point where it occurs on the left-hand side of an assignment
- Identify program points and variable names
- ϕ -functions select proper definitions at control-flow joins

(Un-Conditional) Constant Propagation in SSA

- Perform flow-insensitive analysis on SSA-program
- Domain: $\mathbb{C} := (\text{Vars} \rightarrow \mathbb{Z}_{\perp}^{\top})$
- Transfer functions:

$$[\![\text{;}]^{\#} \mathbb{C} := \mathbb{C}$$

$$[\![x \leftarrow e;]^{\#} \mathbb{C} := \mathbb{C}[x \mapsto [\![e]^{\#} \mathbb{C}]$$

$$[\![x \leftarrow M[e];]^{\#} \mathbb{C} := \mathbb{C}[x \mapsto \top]$$

$$[\![M[e_1] \leftarrow e_2]^{\#} \mathbb{C} := \mathbb{C}$$

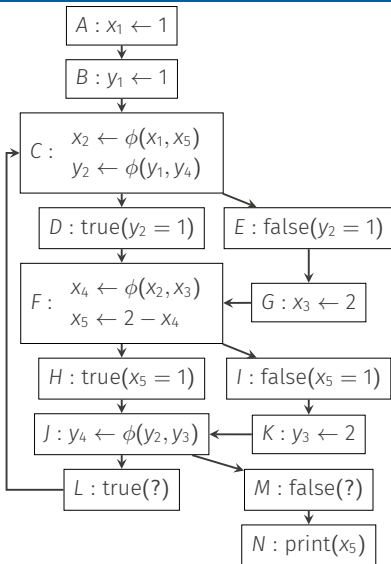
$$[\![x_0 \leftarrow \phi(x_1, \dots, x_n)]^{\#} \mathbb{C} := \mathbb{C}[x_0 \mapsto \bigsqcup_{1 \leq i \leq n} \mathbb{C}(x_i)]$$

- ϕ -functions make join over different reaching definitions **explicit**
- Solve **single** inequality

$$\mathbb{C} \supseteq \bigsqcup_i f_i^{\#} \mathbb{C}$$

by fixpoint iteration

Example



	0	1	2	3
x_1	$\perp$	1	1	1
y_1	$\perp$	1	1	1
x_2	$\perp$	$\perp$	1	$\top$
y_2	$\perp$	$\perp$	1	$\top$
x_3	$\perp$	2	2	2
x_4	$\perp$	$\perp$	$\top$	$\top$
x_5	$\perp$	$\perp$	$\top$	$\top$
y_3	$\perp$	2	2	2
y_4	$\perp$	$\perp$	$\top$	$\top$

Conditional Constant Propagation on SSA

- Consider control flow as well. Perform two analysis in parallel
- Cooperation between two domains:

$$\mathbb{C} := \text{Vars} \rightarrow \mathbb{Z}_{\perp}^{\top} \quad \mathbb{U} := \text{Blocks} \rightarrow \{\perp_U, \top_U\}$$

- $\perp_U$ = unreachable code, $\top_U$ = reachable code
- Two transfer functions per program point i :

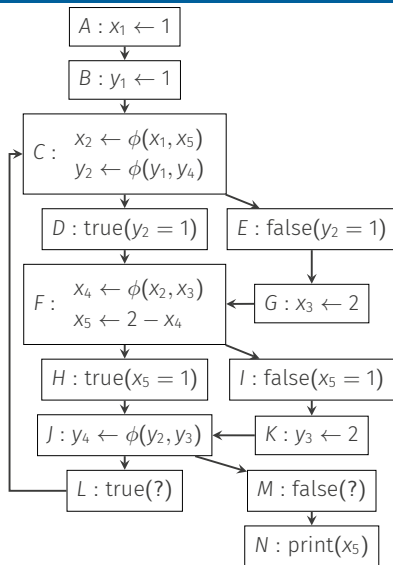
$f_i : \mathbb{C} \times \mathbb{U} \rightarrow \mathbb{C}$ for constant propagation

$g_i : \mathbb{C} \times \mathbb{U} \rightarrow \mathbb{U}$ for reachability

- Solve system of equations

$$\begin{array}{lcl} x & \sqsupseteq & \bigsqcup f_i(x, y) \\ y & \sqsupseteq & \bigsqcup g_i(x, y) \end{array} \quad x \in \mathbb{C}, y \in \mathbb{U}$$

Example



	0	1	2
x_1	$\perp$	1	1
y_1	$\perp$	1	1
x_2	$\perp$	1	1
y_2	$\perp$	1	1
x_3	$\perp$	2	2
x_4	$\perp$	1	1
x_5	$\perp$	1	1
y_3	$\perp$	2	2
A	T_U	T_U	T_U
B	$\perp_U$	T_U	T_U
C	$\perp_U$	T_U	T_U
D	$\perp_U$	T_U	T_U
E	$\perp_U$	$\perp_U$	$\perp_U$
F	$\perp_U$	T_U	T_U
G	$\perp_U$	$\perp_U$	$\perp_U$
H	$\perp_U$	T_U	T_U
I	$\perp_U$	$\perp_U$	$\perp_U$
J	$\perp_U$	T_U	T_U
K	$\perp_U$	$\perp_U$	$\perp_U$
L	$\perp_U$	T_U	T_U
M	$\perp_U$	T_U	T_U
N	$\perp_U$	T_U	T_U

Transfer Functions

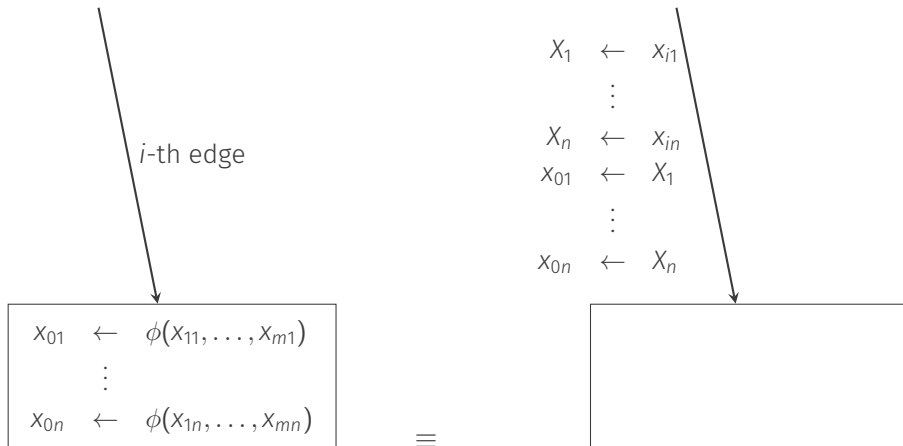
- For constant propagation (functions f_i)

$$\begin{aligned}\llbracket \ell : x \leftarrow e; \rrbracket^\# C, U &:= C[x \mapsto \llbracket e \rrbracket^\# C] \\ \llbracket \ell : x \leftarrow M[e]; \rrbracket^\# C, U &:= C[x \mapsto \top] \\ \llbracket \ell : x_0 \leftarrow \phi(x_1, \dots, x_n) \rrbracket^\# C, U &:= C[x_0 \mapsto \bigsqcup X] \\ X &:= \{x_i \mid U(\text{pred}(\ell, i)) = \top_U\} \\ \llbracket \cdot \rrbracket^\# C, U &:= C\end{aligned}$$

- For reachability (functions g_i)

$$\begin{aligned}\llbracket \ell : \text{true}(e) \rrbracket^\# C, U &:= U \left[\ell \mapsto \begin{cases} \perp_U & \llbracket e \rrbracket^\# C \subseteq 0 \\ \top_U & \text{otherwise} \end{cases} \right] \\ \llbracket \ell : \text{false}(e) \rrbracket^\# C, U &:= U \left[\ell \mapsto \begin{cases} \top_U & 0 \subseteq \llbracket e \rrbracket^\# C \\ \perp_U & \text{otherwise} \end{cases} \right] \\ \llbracket \cdot \rrbracket^\# C, U &:= U\end{aligned}$$

ϕ -functions have semantics

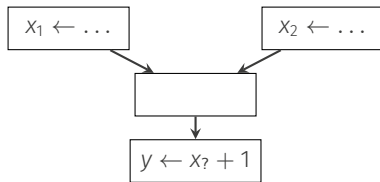


Commonly stated as

All ϕ -functions are evaluated **simultaneously** at the **beginning** of the block

Where to place ϕ -functions?

- ϕ -functions have to be placed such that
 1. SSA program P' has the same semantics as original program P
 2. Every variable has exactly one program point where it is defined



- Observation:
 - First point reached by two different definitions of (non-SSA) variable has to contain a ϕ -function
 - In the SSA-form program, every use is reached by a single unique definition

SSA Construction Algorithm by Cytron et al.

1. Compute dominance tree
2. Compute dominance frontiers for each block
3. Place $v = \Phi(v, \dots, v)$ in the iterated dominance frontier (DF^+) for each definition of a variable v .
 - It's not necessary to explicitly compute DF^+ :
 - Either transitively follow the edges in the dominance frontier graph
 - Or consider that the new definition of v might again provoke new ϕ -functions
4. Rename variables
 - Every use takes lowest definition in the dominance tree
 - Note that ϕ -function uses happen at the end of the predecessors

- “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph”
<https://www.cs.utexas.edu/~pingali/CS380C/2010/papers/ssaCytron.pdf>
- “Simple and Efficient Construction of Static Single Assignment Form”
<https://pp.info.uni-karlsruhe.de/uploads/publikationen/braun13cc.pdf>
- “Constant Propagation with Conditional Branches”
<http://www.cs.wustl.edu/~cytron/531Pages/f11/Resources/Papers/cprop.pdf>